COMP 410 (Sec B)
Parallel and Distributed Computing
Semester Project Report

Group Members

Syed Abdullah 261953683
Zainab Shakeel 271047965
Hafsa Khalil 261945632

Introduction

The MOT dataset is approximately 3GB in size and contains millions of vehicle test records. Processing this volume of data on a single machine results in long query times and poor user responsiveness.This project implements a distributed, parallel data processing system using MPI (Message Passing Interface) to accelerate searching, filtering, and analysis operations. The system supports both CLI and GUI, allowing users to issue queries that are automatically distributed across multiple nodes.

System Architecture

The project uses an MPI-based **Master–Worker Model**:

Roles

Component	Responsibility
Coordinator (Rank 0)	Runs GUI, distributes user queries, aggregates worker results, plots charts
Worker Nodes (Rank 1…N−1)	Load assigned dataset partitions, filter data, compute statistics

Architecture Diagram:

Communication uses **MPI tags** to distinguish commands:

MPI Tag	Operation
TAG_search	Distributed filtering / search
TAG_ANALYZE	Compute pass rate statistics

TAG_EXIT	Graceful shutdown

Module-Wise Overview

File	Description
analytics.py	Functions computing pass rate by age and mileage
config.py	Data paths, limits, MPI message tags
coordinator.py	Full Tkinter GUI + plotting and MPI communication
loader.py	Distributed loading w/ chunk-based cleaning (every 1000 rows)
worker.py	Worker logic: responds to search & analysis requests
main.py	MPI process launcher, initializing coordinator and workers

---

Data Preprocessing & Cleaning

Performed **per chunk** in worker nodes for memory efficiency:

Column	Operation
make, model, test_result	Uppercase, whitespace cleanup
test_date, first_use_date	Parsed to datetime, handling malformed values
test_mileage	Converted to numeric

Filtering rules:

- Ignore **negative or > 60 year old** vehicles (age analytics)
- Bucket mileage in **10,000-mile groups**
- Remove extreme outliers (> 500,000 miles)
- Only include bins with **≥5 test samples** for reliability

Analytics Computation

A. Pass Rate by Age

Formula:

Age = (test_date − first_use_date) / 365.25
Pass_Rate(age) = (#Passed at age) / (Total tests at age) × 100

B. Pass Rate by Mileage

Mileage_Group = (test_mileage // 10000) * 10000
Pass_Rate(mileage_bin) = sum(pass) / count × 100

Worker results are merged using:

Global_Count = Σ local_counts
Global_Sum = Σ local_pass
Global_Pass_Rate = Global_Sum / Global_Count × 100

GUI Features

1. Full-screen Tkinter layout
2. Distributed search with filters:
3. Make & Model
4. First-Use Year
5. Min/Max Mileage
6. Displays top 100 results per query
7. Analytical tab with:
8. Radio button to switch graph type
9. Line charts rendered using Matplotlib

Example Graphs:

- Pass Rate vs Age (vehicle aging performance)
- Pass Rate vs Mileage (wear-and-tear analysis)

Performance Improvements

Feature	Benefit
MPI round-robin file distribution	Balanced workload across workers
Parallel chunk reading	Faster load times even for millions of rows
Filter workers before sending data	Reduced network overhead
Statistical filtering	More accurate visualization

Cluster Computing (Bonus):

Configuration A: Windows-to-Windows MPI Cluster

Step 1: Environment Setup (Both Nodes)

Install Python Dependencies:

pip install mpi4py pandas matplotlib

Install Microsoft MPI:

Download and install msmpisetup.exe and msmpisdk.msi from the official Microsoft website on both machines.

Firewall Configuration:

Go to: *Windows Defender Firewall and Turn Windows Defender Firewall on or off*.

Select Turn off Windows Defender Firewall for both Private and Public networks to allow MPI communication.

Step 2: User Authentication (Coordinator Node)

Register credentials so MPI can access the Worker machine:

mpiexec -register

Enter the Windows username and password of the Worker node when prompted.

Step 3: Start Listener Daemon (Worker Node)

On the Worker machine, run (as Administrator):

smpd -d

Keep this window **open**. Expected output: *smpd listening on port 8677.*

Step 4: Execution (Coordinator Node)

Run the distributed job:

mpiexec -hosts 2 [MASTER_IP] [SLAVE_IP] -n 4 -mapall python main.py

- -hosts 2: Indicates two physical machines
- -n 4: Launches total 4 parallel processes
- -mapall: Uses previously registered credentials

Configuration B: macOS-to-macOS Cluster

Step 1: Environment Setup (Both Nodes)

Install OpenMPI:

brew install open-mpi

Install Python Dependencies:

pip install mpi4py pandas matplotlib

Step 2: Enable Remote Login (Worker Node)

Go to: *System Settings → General → Sharing.* Enable Remote Login
Note the SSH command displayed (e.g., ssh user@192.168.1.5)

Step 3: Establish SSH Connection

Verify that SSH communication works:

ssh [WORKER_USERNAME]@[WORKER_IP]

- Type yes to trust fingerprint

- Enter Worker password
- Use exit to return back to Coordinator terminal

Step 4: Execution (Coordinator Node)

Run the distributed job:

mpirun -host [MASTER_IP],[SLAVE_IP] -np 4 python main.py

- -host: Comma-separated list of cluster node IPs
- -np 4: Launches total 4 parallel processes

Conclusion

The project successfully demonstrates:

- Parallel data processing
- MPI Master/Worker architecture
- Efficient data decomposition
- Distributed query execution
- GUI-integrated analytics and graphing